

Walrus Weights

Day 3 solution walkthrough

Competitive Programming Bootcamp

Statement excerpts:

- ▶ *“as close as possible to \$1\,000\$ kg”*
- ▶ *“Wallace will pick the greater one”*

Programming interpretation:

- ▶ Each plate can be used or skipped, so this is subset-sum DP.
- ▶ Track which total weights up to 2000 are reachable.
- ▶ Search outward from 1000, checking the higher tie first.

Textbook connection: CP4 3.5 g. DP Level 2 / Subset Sum

Core technique: Subset-sum DP around a target

- ▶ Choose any subset of weight plates.
- ▶ The target total is as close as possible to 1000.
- ▶ If tied, choose the larger total.

- ▶ This is subset sum rather than an optimization over order.
- ▶ Only totals from 0 to 2000 matter for closeness to 1000.
- ▶ A boolean DP array records which totals are reachable.

- ▶ Initialize $dp[0] = \text{True}$.
- ▶ For each plate weight, scan current totals backward.
- ▶ If $dp[\text{current}]$ is true, set $dp[\text{current} + \text{weight}]$ to true.
- ▶ After processing plates, search outward from 1000.
- ▶ Check the higher value before the lower value at each distance.

- ▶ Backward scanning prevents reusing the same plate in one iteration.
- ▶ The first reachable total found by outward search has minimum absolute distance.
- ▶ Checking higher first enforces the tie-breaking rule.

- ▶ After each plate, $dp[w]$ is true exactly when some subset of processed plates sums to w .
- ▶ The transition adds the current plate to every previously reachable total.
- ▶ The outward search examines totals in nondecreasing distance from 1000, so the first reachable answer is optimal.

Complexity: $O(N * 2000)$ time and $O(2000)$ memory.

- ▶ The 2000 cap is safe because anything farther than 1000 from the target cannot beat a value in $[0, 2000]$.
- ▶ This is a compact one-dimensional 0/1 DP.

Source lines: walrusweights.py:40-60

```
import sys

def main():
    input = sys.stdin.buffer.readline

    n = int(input())

    weights = []

    for _ in range(n):
        weights.append(int(input()))

    dp = [False] * 2001

    dp[0] = True
```

What this chunk does

- ▶ The list stores all plate weights from the input.
- ▶ $dp[w]$ means an exact total of w can be made.
- ▶ A total of 0 is always reachable before choosing any plates.

Source lines: walrusweights.py:62-75

```
for weight in weights:
    for current in range(2000 - weight, -1, -1):
        if dp[current]:
            dp[current + weight] = True
```

What this chunk does

- ▶ For each plate, the DP scans totals backward.
- ▶ Backward order guarantees each plate contributes at most once.
- ▶ If current is reachable, current + weight becomes reachable.

Source lines: walrusweights.py:77-99

```
for distance in range(1001):  
  
    higher = 1000 + distance  
    lower = 1000 - distance  
  
    if higher <= 2000 and dp[higher]:  
        print(higher)  
        return  
  
    if lower >= 0 and dp[lower]:  
        print(lower)  
        return  
  
main()
```

What this chunk does

- ▶ distance grows from 0 outward, so the first reachable answer is closest.
- ▶ The higher value is checked before the lower value to satisfy the tie rule.
- ▶ Once a reachable total is found, it is printed immediately.

Solution file: `practice_contests/day_3/walrusweights.py`

Problem: <https://open.kattis.com/problems/walrusweights>

Use this deck with the source code open beside it. The slides explain the reasoning; the code shows the exact input handling and output formatting.